

Project 5 – PowerShell IT Support Automation Toolkit

Project Overview

This project showcases how PowerShell can be used to perform common IT Support and Help Desk tasks in a Windows environment. Throughout the lab, PowerShell was used to gather system information, manage local user accounts, troubleshoot Windows services, perform network diagnostics, and investigate Windows Event Logs.

The goal of this project was to develop practical PowerShell skills that can improve troubleshooting efficiency and support day-to-day administrative tasks commonly performed by IT Support professionals.

By completing this lab, I gained hands-on experience using PowerShell as a troubleshooting and administration tool while strengthening my understanding of Windows operating systems, networking, and user management.

Project Objectives

- Collect system information using PowerShell
- Manage local user accounts
- Troubleshoot Windows services
- Perform network diagnostics
- Investigate Windows Event Logs
- Improve troubleshooting efficiency through automation
- Develop practical PowerShell skills for IT Support roles

Environment and Tools Used

- Windows 10 Home
- Windows PowerShell 5.1
- Windows Event Viewer
- Local User and Group Management
- TCP/IP Networking
- Windows Services

Phase 1 – System Information Collection

Scenario

A user reported that their workstation was running slowly. Before performing troubleshooting activities, basic system information was collected to establish a baseline of the device configuration.

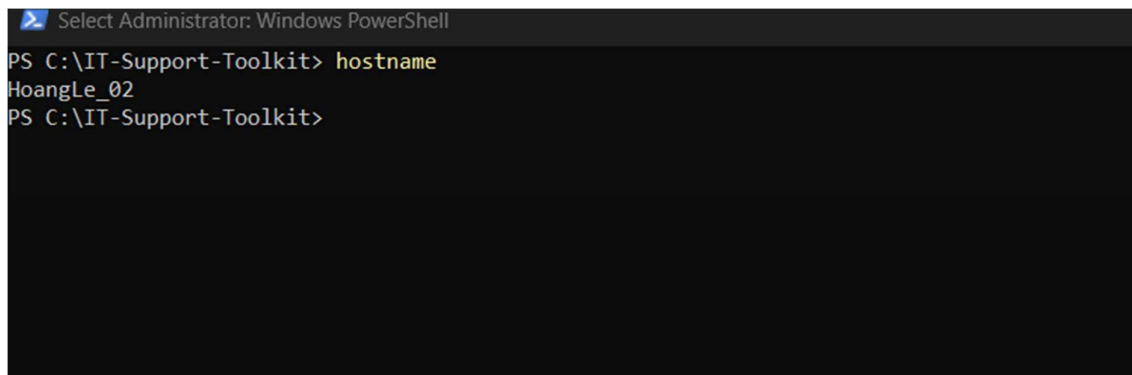
Objective

Use PowerShell to gather key system information including the computer name, logged-in user account, Windows version, network configuration, and storage status.

Steps Taken

Step 1 – Identify the Computer Name

I used the **hostname** command to identify the workstation name. Knowing the computer name is important when providing remote support, managing devices, or locating a machine within an organization.

A screenshot of a Windows PowerShell terminal window. The title bar at the top reads "Select Administrator: Windows PowerShell". The terminal shows a command prompt with the path "PS C:\IT-Support-Toolkit>". The user has entered the command "hostname" in yellow text. The output "HoangLe_02" is displayed in white text on the line below. The prompt "PS C:\IT-Support-Toolkit>" is shown again on the next line, indicating the command has been executed.

```
Select Administrator: Windows PowerShell
PS C:\IT-Support-Toolkit> hostname
HoangLe_02
PS C:\IT-Support-Toolkit>
```

Step 2 – Identify the Logged-in User

I ran the **whoami** command to determine which user account was currently signed in. This information is useful when verifying user identity and troubleshooting account-related issues.

```
Administrator: Windows PowerShell
PS C:\IT-Support-Toolkit> hostname
Hoangle_02
PS C:\IT-Support-Toolkit> whoami
hoangle_02\hle22
PS C:\IT-Support-Toolkit>
```

Step 3 – Check the Windows Version

I used the following command to verify the installed Windows edition:

Command

```
Get-ComputerInfo | Select WindowsProductName
```

The output confirmed that the device was running Windows 10 Home.

```
PS C:\IT-Support-Toolkit> Get-ComputerInfo | Select WindowsProductName

WindowsProductName
-----
Windows 10 Home

PS C:\IT-Support-Toolkit>
```

Step 4 – Review IPv4 Network Configuration

I used the following command to review the IPv4 configuration of the available network adapters:

Command

```
Get-NetIPAddress -AddressFamily IPv4
```

While reviewing the results, I noticed that the adapters were assigned APIPA addresses (169.254.x.x), which typically indicate that the device was unable to obtain an IP address from a DHCP server.

```
Windows 10 Home

PS C:\IT-Support-Toolkit> Get-NetIPAddress -AddressFamily IPv4

IPAddress      : 169.254.190.151
InterfaceIndex : 23
InterfaceAlias : Ethernet 2
AddressFamily  : IPv4
Type           : Unicast
PrefixLength   : 16
PrefixOrigin   : WellKnown
SuffixOrigin   : Link
AddressState   : Tentative
ValidLifetime  : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource   : False
PolicyStore    : ActiveStore

IPAddress      : 169.254.246.83
InterfaceIndex : 8
InterfaceAlias : Wi-Fi 2
AddressFamily  : IPv4
Type           : Unicast
PrefixLength   : 16
PrefixOrigin   : WellKnown
SuffixOrigin   : Link
AddressState   : Tentative
```

Step 5 – Check Storage Health and Disk Space

I ran the following command to review storage information:

Get-Volume

The output displayed the available volumes, file system types, health status, and remaining disk space. I confirmed that the system drive was healthy and had sufficient free space available.

```
PS C:\IT-Support-Toolkit> Get-Volume
```

DriveLetter	FriendlyName	FileSystemType	DriveType	HealthStatus	OperationalStatus	SizeRemaining	Size
C	SYSTEM	FAT32	Fixed	Healthy	OK	201.86 MB	256 MB
	RECOVERY	NTFS	Fixed	Healthy	OK	71.18 MB	1000 MB
	OS	NTFS	Fixed	Healthy	OK	67.99 GB	237.03 GB
	MYASUS	FAT32	Fixed	Healthy	OK	140.35 MB	196 MB

```
PS C:\IT-Support-Toolkit>
```

Validation

I successfully collected key system information using PowerShell, including the computer name, active user account, Windows version, network configuration, and storage details. This information provided a useful baseline for future troubleshooting and support activities.

Phase 2 – Local User Account Management

Scenario

A new employee required a local account on a workstation. Before creating the account, it was necessary to review the existing local users and verify the current account configuration. The account lifecycle was then tested by creating, disabling, re-enabling, and finally removing a user account.

Objective

The objective of this phase was to gain hands-on experience managing local user accounts with PowerShell and understand common account administration tasks performed by IT Support technicians.

Steps Taken

Step 1 – Review Existing Local User Accounts

I began by reviewing the local user accounts configured on the workstation. This allowed me to identify built-in Windows accounts as well as the active user account currently present on the system.

Command

Get-LocalUser

```
PS C:\IT-Support-Toolkit> Get-LocalUser

Name                Enabled Description
-----
Administrator       False Built-in account for administering the computer/domain
DefaultAccount       False A user account managed by the system.
Guest                False Built-in account for guest access to the computer/domain
hle22                True
WDAGUtilityAccount   False A user account managed and used by the system for Windows Defender Application

PS C:\IT-Support-Toolkit>
```

Step 2 – Create a Test User

To simulate a new employee onboarding process, I created a local account named **TestUser**. Creating local accounts is useful when preparing standalone workstations, lab environments, or devices that are not joined to a domain

Command

New-LocalUser -Name "TestUser" -NoPassword

```
PS C:\IT-Support-Toolkit> New-LocalUser -Name "TestUser" -NoPassword
```

Name	Enabled	Description
TestUser	True	

Step 3 – Disable a User Account

Next, I disabled the TestUser account to simulate a temporary account suspension. This is a common administrative task during employee offboarding, security investigations, or extended leave periods.

After verifying the account status, I confirmed that sign-in access had been successfully disabled.

Command

Disable-LocalUser -Name "TestUser"

```
PS C:\IT-Support-Toolkit> Disable-LocalUser -Name "TestUser"
```

```
PS C:\IT-Support-Toolkit> Get-LocalUser
```

Name	Enabled	Description
Administrator	False	Built-in account for administering the computer/domain
DefaultAccount	False	A user account managed by the system.
Guest	False	Built-in account for guest access to the computer/domain
hle22	True	
TestUser	False	
WDAGUtilityAccount	False	A user account managed and used by the system for Windows Defender Application

Step 4 – Re-enable the User Account

Once the account had been disabled, access was restored by re-enabling it. This step simulated a scenario where a user returns from leave or an account is reactivated after being disabled by mistake.

A verification check confirmed that the account was active again and available for use.

Command

Enable-LocalUser -Name "TestUser"

```
PS C:\IT-Support-Toolkit> Enable-LocalUser -Name "TestUser"
PS C:\IT-Support-Toolkit> Get-LocalUser

Name                Enabled Description
-----
Administrator       False Built-in account for administering the computer/domain
DefaultAccount       False A user account managed by the system.
Guest                False Built-in account for guest access to the computer/domain
hle22                True
TestUser             True
WDAGUtilityAccount   False A user account managed and used by the system for Windows Defender Application

PS C:\IT-Support-Toolkit>
```

Step 5 – Remove a Local User Account

Once testing was completed, the account was removed from the workstation. Removing unused accounts is an important security practice because it reduces unnecessary access and helps maintain a clean user environment.

A final verification confirmed that the TestUser account no longer appeared in the local user list.

Command

Remove-LocalUser -Name "TestUser"

```
PS C:\IT-Support-Toolkit> Remove-LocalUser -Name "TestUser"
PS C:\IT-Support-Toolkit> Get-LocalUser

Name                Enabled Description
-----
Administrator       False Built-in account for administering the computer/domain
DefaultAccount       False A user account managed by the system.
Guest                False Built-in account for guest access to the computer/domain
hle22                True
WDAGUtilityAccount   False A user account managed and used by the system for Windows Defender Application

PS C:\IT-Support-Toolkit>
```

Validation

I successfully completed the full local account lifecycle using PowerShell. During this phase, I reviewed existing user accounts, created a new local user, disabled and re-enabled the account, and finally removed it from the workstation. Each action was verified to ensure the expected changes were applied successfully. This phase strengthened my understanding of user account administration tasks commonly performed by IT Support technicians.

Phase 3 – Service Troubleshooting (Print Spooler)

Scenario

A user reported that they were unable to print documents. To investigate the issue, I reviewed the status of the Print Spooler service, simulated a service interruption, and then restored the service to verify that printing functionality could be recovered.

Objective

The objective of this phase was to understand how Windows services can be managed with PowerShell and gain hands-on experience troubleshooting service-related issues commonly encountered by IT Support technicians.

Steps Taken

Step 1 – Check Print Spooler Service Status

I began by checking the current status of the Print Spooler service. Since this service manages print jobs sent to printers, verifying its status is one of the first troubleshooting steps when users report printing issues.

The output confirmed that the service was running normally.

Command

Get-Service Spooler

```
PS C:\IT-Support-Toolkit> Get-service Spooler

Status  Name      DisplayName
-----
Running Spooler   Print Spooler

PS C:\IT-Support-Toolkit>
```

Step 2 – Stop the Print Spooler Service

To simulate a real-world printing issue, I stopped the Print Spooler service. This allowed me to observe how service interruptions can affect user functionality and provided an opportunity to practice troubleshooting procedures.

A verification check confirmed that the service status changed to **Stopped**.

Command

Stop-Service Spooler

```
PS C:\IT-Support-Toolkit> Stop-Service Spooler
PS C:\IT-Support-Toolkit> Get-Service Spooler

Status      Name      DisplayName
-----
Stopped     Spooler    Print Spooler

PS C:\IT-Support-Toolkit>
```

Step 3 – Start the Print Spooler Service

After confirming the service had stopped, I restarted it to restore normal operation. Restarting Windows services is a common troubleshooting technique used to resolve temporary service failures and application issues.

A final verification confirmed that the service returned to a **Running** state and was operating normally again.

Command

Start-Service Spooler

```
PS C:\IT-Support-Toolkit> Start-Service Spooler
PS C:\IT-Support-Toolkit> Get-Service Spooler

Status      Name      DisplayName
-----
Running     Spooler    Print Spooler

PS C:\IT-Support-Toolkit>
```

Validation

I successfully completed the troubleshooting process by checking the Print Spooler service, stopping it to simulate a printing issue, and starting it again to restore functionality. After each action, I verified the service status to ensure the expected changes had taken place. This confirmed that PowerShell can be used effectively to diagnose and resolve common Windows service issues.

Phase 4 – Network Diagnostics

Scenario

A user reported that they were unable to access websites or company resources. To investigate the issue, I used several PowerShell networking commands to review the workstation's network configuration, test Internet connectivity, verify DNS functionality, and identify the default gateway.

Objective

The objective of this phase was to gain hands-on experience using PowerShell for network troubleshooting and understand how common connectivity issues can be identified and analyzed in an IT Support environment.

Steps Taken

Step 1 – Review IPv4 Network Configuration

I started by reviewing the IPv4 configuration of the available network adapters. During the review, I noticed that some adapters were assigned APIPA addresses (169.254.x.x), which typically indicate that a device was unable to obtain an IP address from a DHCP server.

This exercise helped me understand how PowerShell can be used to quickly identify addressing and connectivity issues.

Command

Get-NetIPAddress -AddressFamily IPv4

```
PS C:\IT-Support-Toolkit> Get-NetIPAddress -AddressFamily IPv4

IPAddress      : 169.254.190.151
InterfaceIndex : 23
InterfaceAlias : Ethernet 2
AddressFamily  : IPv4
Type           : Unicast
PrefixLength   : 16
PrefixOrigin   : WellKnown
SuffixOrigin   : Link
AddressState   : Tentative
ValidLifetime  : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource   : False
PolicyStore    : ActiveStore

IPAddress      : 169.254.246.83
InterfaceIndex : 8
InterfaceAlias : Wi-Fi 2
AddressFamily  : IPv4
Type           : Unicast
PrefixLength   : 16
```

Step 2 – Test Network Connectivity

Next, I tested connectivity to an external host using PowerShell. The results confirmed that the workstation was able to communicate successfully with Google and that Internet access was available.

The output also identified the active source IP address being used for network communication, demonstrating that a working connection existed despite other adapters displaying APIPA addresses.

Command

Test-NetConnection google.com

```
PS C:\IT-Support-Toolkit> Test-NetConnection google.com

ComputerName           : google.com
RemoteAddress          : 142.250.195.142
InterfaceAlias         : Wi-Fi
SourceAddress          : 172.20.10.10
PingSucceeded          : True
PingReplyDetails (RTT) : 31 ms

PS C:\IT-Support-Toolkit>
```

Step 3 – Test DNS Resolution

To confirm that name resolution was functioning correctly, I performed a DNS lookup for google.com.

The command successfully returned both IPv4 and IPv6 addresses, confirming that the workstation could resolve domain names into IP addresses and communicate with DNS services correctly.

Command

Resolve-DnsName google.com

```
PS C:\IT-Support-Toolkit> Resolve-DnsName google.com
```

Name	Type	TTL	Section	IPAddress
google.com	AAAA	101	Answer	2404:6800:4006:801::200e
google.com	A	268	Answer	172.217.25.206

```
PS C:\IT-Support-Toolkit>
```

Step 4 – Check the Default Gateway

As a final step, I reviewed the default route configuration to identify the gateway responsible for forwarding traffic outside the local network.

The results confirmed that a valid default gateway was configured and available, providing a path for communication with external networks such as the Internet.

Command

Get-NetRoute -DestinationPrefix "0.0.0.0/0"

```
PS C:\IT-Support-Toolkit> Get-NetRoute -DestinationPrefix "0.0.0.0/0"
```

ifIndex	DestinationPrefix	NextHop	RouteMetric	ifMetric	PolicyStore
9	0.0.0.0/0	172.20.10.1	0	35	Active

```
PS C:\IT-Support-Toolkit>
```

Validation

I successfully used PowerShell to review network configuration details, test Internet connectivity, verify DNS resolution, and identify the default gateway. This phase strengthened my understanding of network troubleshooting techniques and demonstrated how PowerShell can be used to diagnose common connectivity issues in an IT Support environment.

Phase 5 – Event Log Investigation

Scenario

A user reported that their workstation had been experiencing unexpected errors and performance issues. To investigate the problem, I reviewed recent Windows System Event Logs and analyzed warning and error events that could indicate underlying system issues.

Objective

The objective of this phase was to gain hands-on experience using PowerShell to investigate Windows Event Logs and understand how event data can be used during troubleshooting and incident investigation.

Steps Taken

Step 1 – View Recent System Event

I began by reviewing the most recent entries in the Windows System Event Log. This provided a general overview of recent operating system activity, including informational events, warnings, and system-related messages.

While reviewing the output, I identified several warning events generated by **Win32k** alongside normal operating system activity. This demonstrated how Event Logs can provide valuable insight into system behavior during troubleshooting.

Command

```
Get-EventLog -LogName System -Newest 10
```

```
PS C:\IT-Support-Toolkit> Get-EventLog -LogName System -Newest 10
```

Index	Time	EntryType	Source	InstanceID	Message
152009	Jun 07 16:12	Information	Microsoft-Windows...	1	The description for Event ID '1...
152008	Jun 07 15:12	Information	Microsoft-Windows...	1	The description for Event ID '1...
152007	Jun 07 15:05	Information	Microsoft-Windows...	566	The description for Event ID '5...
152006	Jun 07 15:05	Information	Microsoft-Windows...	507	The description for Event ID '5...
152005	Jun 07 15:05	Warning	Win32k	2147484349	Power Manager has not requested...
152004	Jun 07 15:05	Warning	Win32k	2147484349	Power Manager has not requested...
152003	Jun 07 13:29	Information	EventLog	2147489661	The system uptime is 363470 sec...
152002	Jun 07 03:29	Information	Microsoft-Windows...	566	The description for Event ID '5...
152001	Jun 07 03:29	Information	Microsoft-Windows...	506	The description for Event ID '5...
152000	Jun 07 03:29	Information	Microsoft-Windows...	566	The description for Event ID '5...

```
PS C:\IT-Support-Toolkit>
```

Step 2 – Filter Warning Events

To focus on events that may require further attention, I filtered the log results to display only warning-level entries.

The filtered output highlighted several warnings related to power management, USB power usage, NTP client configuration, and system resource management. By narrowing the results to warning events, it became easier to identify issues that could potentially affect system stability or performance.

Command

```
Get-EventLog -LogName System -EntryType Warning -Newest 10
```

```
PS C:\IT-Support-Toolkit> Get-EventLog -LogName System -EntryType Warning -Newest 10
```

Index	Time	EntryType	Source	InstanceID	Message
152005	Jun 07 15:05	Warning	Win32k	2147484349	Power Manager has not requested...
152004	Jun 07 15:05	Warning	Win32k	2147484349	Power Manager has not requested...
151998	Jun 07 03:29	Warning	Win32k	2147484348	Power Manager has requested sup...
151997	Jun 07 03:29	Warning	Win32k	2147484348	Power Manager has requested sup...
151970	Jun 07 02:37	Warning	Microsoft-Windows...	134	NtpClient was unable to set a m...
151969	Jun 07 02:37	Warning	Win32k	2147484349	Power Manager has not requested...
151968	Jun 07 02:37	Warning	Win32k	2147484349	Power Manager has not requested...
151967	Jun 06 14:57	Warning	Microsoft-Windows...	196	USB device draining system powe...
151946	Jun 06 14:36	Warning	Win32k	2147484348	Power Manager has requested sup...
151945	Jun 06 14:36	Warning	Win32k	2147484348	Power Manager has requested sup...

```
PS C:\IT-Support-Toolkit>
```

Step 3 – Review Error Events

As a final step, I investigated recent error-level events recorded in the System Log.

Several errors were identified from sources including **Volsnap**, **DCOM**, **Service Control Manager**, and **VBoxNetLwf**. These events were associated with volume shadow copy operations, service failures, communication issues, and VirtualBox networking components.

Reviewing error events provided a better understanding of how Windows records system failures and highlighted the importance of Event Logs during root cause analysis and troubleshooting activities.

Command

```
Get-EventLog -LogName System -EntryType Error -Newest 10
```

```

PS C:\IT-Support-Toolkit> Get-EventLog -LogName System -EntryType Error -Newest 10

Index Time           EntryType Source                InstanceID Message
-----
151979 Jun 07 02:40 Error volsnap 3221618724 The shadow copies of volume C: ...
151882 Jun 05 12:30 Error DCOM 10010 The description for Event ID '1...
151881 Jun 05 12:09 Error Microsoft-Windows... 20 Installation Failure: Windows f...
151875 Jun 05 12:09 Error DCOM 10010 The description for Event ID '1...
151866 Jun 05 12:00 Error DCOM 10010 The description for Event ID '1...
151865 Jun 05 12:00 Error DCOM 10010 The description for Event ID '1...
151844 Jun 05 10:52 Error Service Control M... 3221232472 The Steam Client Service servic...
151843 Jun 05 10:52 Error Service Control M... 3221232481 A timeout was reached (60000 mi...
151836 Jun 05 10:51 Error Microsoft-Windows... 10317 Miniport Microsoft Wi-Fi Direct...
151832 Jun 05 10:51 Error VBoxNetLwf 3221487628 The driver detected an internal...

PS C:\IT-Support-Toolkit>

```

Validation

I successfully reviewed recent System Event Logs, filtered warning events, and analyzed error-level entries using PowerShell. Throughout the investigation, I identified several event sources associated with operating system activity, power management, service failures, and application-related errors. This phase demonstrated how Event Logs can be used to investigate system issues and support troubleshooting efforts in a real-world IT Support environment.

Outcome and Validation

All project objectives were completed successfully. Throughout this lab, I used PowerShell to collect system information, manage local user accounts, troubleshoot Windows services, perform network diagnostics, and investigate Windows Event Logs.

Each task was completed and validated using PowerShell commands, allowing me to verify system configurations, account changes, service states, network connectivity, and event log data. The project demonstrated how PowerShell can be used as a practical administration and troubleshooting tool within an IT Support environment.

What I Learned

Through this project, I gained hands-on experience using PowerShell to perform common IT Support and Help Desk tasks. I learned how to gather system information, manage local user accounts, troubleshoot Windows services, diagnose network connectivity issues, and analyze Windows Event Logs.

This project also improved my confidence working with the Windows command-line environment and showed me how PowerShell can simplify administrative tasks while supporting efficient troubleshooting in real-world IT Support scenarios.